School of Computing and Creative Technologies

# Assessment Brief

## Submission details

| | |
|---|---|
| **Module title**: | Advanced Software Development |
| **Module code**: | UFCF8S-30-2 |
| **Module Leader:** | Barkha Javed and Majid Mumtaz |
| **Assessment title:** | Portfolio and Viva/Demo |
| **Year:** | 2025-26 |
| **Assessment type**: | Portfolio |
| **Assessment weighting:** | 100% of total module mark |
| **Module learning outcomes assessed by this task:** | 1. Analyse problems in order to identify software-solution approaches and requirements for computer-based software-intensive systems.<br>2. Compare and contrast software development methodologies and choose one suitable for a given application.<br>3. Design, implement, test and manage reasonably sized software system considering database and GUI components.<br>4. Discuss the need for security in the context of system development. |
| **Use of AI in assessment:** | You can use Generative AI in this assignment for generating code snippets. You may adapt the code snippets if you fully understand how, it works and can reproduce it if you are asked to do so without Generative AI. You CANNOT use generative AI for design. |
| **Guidance on Referencing (inc AI):** | Please note that the aim of referencing is to demonstrate you have read and understood a range of sources to evidence your key points. You need to list the references consistently and in such a way as to ensure the reader can follow up on the sources for themselves. You should use UWE Bristol Harvard style for referencing. Below are links to referencing guide and reference to generative AI.<br>Referencing - Study skills \| UWE Bristol<br>Using generative AI at UWE Bristol - Study skills \| UWE Bristol |

## Submission and feedback dates

| | |
|---|---|
| **Submission deadline:** | Before 14:00 on **23/04/2026**<br>Is **NOT eligible** for 48-hour late submission window<br><br>All times are 24-hour clock, current local time (at time of submission) in the UK. |
| **Submission format:** | One PDF/word document for all diagrams; source code in a ZIP file; testing document (separate OR test cases can be added to design document as well) |
| **Marks and Feedback due on:** | 20 working days after the submission date |
| **Marks and Feedback will be provided via:** | Blackboard |

**Submission Notes**: Please submit ONE Zip file containing
(1) A PDF document containing Use Case, Class and Sequence diagrams.
(2) Please submit your code, a PDF report on Agile practices, Test cases used in your program in the form of a table, suitable program running screenshots capturing success/failure of automated testing scenarios.

**Please note that all the members of a group must be present during the demonstration sessions, and all members must upload portfolio individually.**

## Completing your assessment

**What am I required to do on this assessment?**

This is a **GROUP assignment** and to be completed in groups of up to **five (5) students**. As a group you have been asked to analyse the following case study, perform requirement analysis, design, implement and test the above system. Please note **responsibilities for each group member** should be clearly defined as marks will be adjusted based on your individual as well as group contributions. Therefore, you must maintain **evidence logs of your contributions** to the project e.g., through regular Git commits. **Please note that group with less than 4 members is not permitted.**
More specifically:

- The assessment requires you to analyse, design and implement an object-oriented system (a desktop application) based on the following case study.
- You will analyse requirements and produce detailed object models and designs from system requirements.
- Use the modelling concepts provided by Unified Modelling Language (UML).
- You'll have a question and answers session with your tutor(s) to explain your analysis and design as a group.
- Using your design, you will implement the software and perform software testing by applying a combination of manual and automated testing tools.
- You will apply software methodology on the project.
- Finally, you will demonstrate your system and answer questions about your design, software implementation and the use of your software development approach as a group.

### Case Study: Paragon Apartment Management System (PAMS)

**Problem statement:** The Paragon apartment management company has a multi-location structure with offices spread across the UK, including Bristol, Cardiff, London, and Manchester. The company faces a series of interconnected operational challenges stemming from its decentralized structure, which complicates coordination and fosters inconsistent data handling practices. A heavy reliance on manual, paper-based processing for critical functions like tenant applications and rent collection creates inefficiencies and increases the risk of errors. Compounding these issues are significant access and security problems, as the lack of clear role-based permissions exposes sensitive tenant and financial data to potential unauthorized access. Furthermore, tenant records are not synchronized across cities, leading to duplication and data inconsistency, while the existing system's inability to scale results in performance issues during peak hours. This outdated infrastructure also fails to provide management with the real-time reporting needed for insights into occupancy or financial performance, and its poor usability necessitates extensive staff training. The key challenge, therefore, is to design and develop a consolidated, scalable, and secure Software solution that is intuitive to use and capable of supporting complex, multi-city operations with robust role-based access control. The Paragon company has requested a Software Development team (your IT company) to provide a solution. They require a software solution from your Development Team to ensure stakeholder requirements are met during the design and implementation phases.

Their initial list of essential components includes Account / User Management, Tenant Management, Apartment Management, Payment & Billing, Report Generation, and maintenance.

Furthermore, they want a robust solution including the following key components:

**Account /User Management component:**
Account /user management component is defined by four core roles:

- **Front-desk Staff** can register new tenants and access information to handle tenant inquiries. For the registration of a new tenant, the front-desk staff requires tenant's National Insurance (NI) number, name, phone number, email, occupation, references, apartment requirements (such as two-bedroom house), lease period, and core fields that you think are important. The front-desk staff are also responsible for registering and tracking maintenance requests and complaints.
- **Finance Manager** manages payments, invoices, late payments and financial reports.
- **Maintenance Staff** manages maintenance requests and logs in all the data once issues are resolved.

- **Administrators** have full system access to a particular location to manage all user accounts, manage apartments, generate reports to assigned location, and track lease agreements such as end date of a lease.
- **Manager** can oversee all apartment occupancy levels and generate various performance reports according to location. Manager can also expand the business in other cities.

**Tenant Management component** core functions include maintaining tenant records by adding, updating, and removing information, as well as tracking associated lease agreements, payment histories, and complaint logs. In some situations, tenants can also request to leave before the end of contract (they must give 1 month notice and as penalty pay 5% of their monthly rent).

**Apartment management component** core functions should include apartment registration with detailed attributes such as location, type, monthly rent, number of rooms etc.; assigning apartment to tenant to track occupancy; and managing the lifecycle of maintenance requests.

**The Payment & Billing Component manages** invoices, payments, late payments, and generates notification if the payment is late. Note that there is no need to integrate a payment processing system; the goal is only to emulate the payment process by generating the billing/dues receipts.

**The reporting component** provides essential operational insights through three key functions: generating occupancy reports filtered by apartment or city, producing financial summaries that compare collected versus pending rents, and tracking maintenance costs for effective budget management.

**The Maintenance component** Manages any maintenance issues that are reported by tenants. The maintenance staff investigates the issues, prioritises them, checks worker availability, communicates maintenance dates and times to tenants, resolves the issues, and records their resolution in the system. Staff also log the time taken to fix each issue and the associated costs.

Other suitable components are welcome which you think can add value to system & operation management. Also, your solution should include non-functional requirements (security, efficiency and scalability) features besides core functionalities.

**More specifically there are following tasks:**

You are asked to produce the following deliverables for the scenario given above.

**Task 1 (Portfolio 75%)**

**Element 1 (System Design- 35 marks)**

Element 1 requires you to produce (i) a use case diagram to capture the functionality for the system to be built. Your use case diagram should be self-explanatory, and you are not required to provide use case descriptions (ii) a class diagram to meet all the requirements captured in the use case diagram and (iii) at least three sequence diagrams for some use cases that adds information. Each sequence diagram should represent a single use case.

**Element 2 (System Development and Methodology- 30 marks)**

Develop the system using a suitable Object-Oriented Programming language (e.g., Python as we'll use python for this module) and Database(s) considering all the functionalities described above.

- You should design and implement suitable database(s) and fill it with suitable mock data for testing and demonstration purposes.
- You should be creative and come up with your own User Interface Design for different above-mentioned GUI needs.
- You should also implement non-functional requirements (e.g., security measures) to a good standard.
- Provide evidence of contribution and methodology used in the project. As evidence, you can write a joint report outlining the steps that you followed to develop the system in relation to an Agile development episode. Also, explicitly specify/show the contribution in the project.

**Please note that you are required to develop a desktop application. Website development is NOT permitted.**

**Element 3 (Testing- 10 marks)**

Testing is an essential part of the software development process. You should use test strategies (manual and automated) to test your implemented system thoroughly. You should identify suitable test cases for all the classes implemented in your system and test individual component as well as the integrated system. When you test your code, you should make sure that your program does not allow bad data to be stored into your objects. Deliberately input out-of-range or incorrect data into your program and try to make it fail. Check if you can insert invalid values into the member variables. One example test case is shown below. You should provide evidence (such as screenshots) of test cases. You can show screenshots in the actual output column (See sample Test Case table below).

| Test Case # | Description | Test dataset/Input | Expected output | Actual output |
|---|---|---|---|---|
| 1 | Front-desk staff is able to add 2 new tenants | Adding city, apartment name/number, NI number, tenants name, occupation, references, lease period, start contract date, end contract | Shows new tenants added. | As expected |

| | | | | |
|---|---|---|---|---|
| | | date, deposit amount, monthly rent | | |

**Task 2 (Viva and Demo 25%)**

The second assessment task is viva and demo of the design, methodology, and overall implemented software solution. The demo will take place during your practical session time, following the submission of your portfolio. The demo/viva will provide you with the opportunity to present and explain your work based on which the quality of your portfolio will be determined. In the demo session, you are expected to show quality of software features, explain the applied software methodology, clarify individual contributions and the overall evaluation of the project.

Note: **All the members of the group must be present during the demonstration session. Passing both Task 1 and Task 2 is mandatory to complete this module.**

Your lab tutor will play role of representative of PAMS. If you have questions about this assignment, please discuss with your lab tutors. You may also post questions for your lab tutors on the discussion board in Blackboard. You can find the discussion board from the front page of the module Blackboard pages and use the forum under the title "Portfolio Queries".

## Deliverables

You must use the Blackboard electronic submission system to submit your work. Each student will have to upload complete package individually. Electronically submitted deliverables include:

Each student needs to upload the following deliverables in one .zip file. The naming standard of the zip file is GroupNo_1234567.zip where 1234567 is the student Id.

1. For **Element 1**: One PDF document for all the diagrams. **Important Note: Please DO NOT submit any other file format. All diagrams (use case, class, and sequence) MUST be submitted in ONE PDF file. Individual diagrams' submission will NOT be accepted.**
2. For **Element 2**, (i) A software system based on the specifications in previous section, with source code in ZIP file (using e.g., Zip). This should contain all the files and folders for the full working system. All program files must have student ID and student name who has written the code. You should also include list of additional Python packages (e.g., using pip list command) needed to run your software. Please do not include python distribution folder in your ZIP file. Provide a text file with instructions on how to start and use your software system e.g., any passwords etc (ii) relevant Database(s) (e.g. MySQL or MongoDB) dump. **Evidence of Methodology and contribution in a PDF file.**
3. For **Element 3**, evidence of testing e.g., test cases, unit tests and outputs. These can be included in a separate PDF document

### Instructions for submission

You must submit your work *before* the stated deadline by electronic submission through Blackboard.

- Multiple submissions can be made to the portal, but only the final one will be accepted and marked.
- **It is your responsibility to submit deliverables in a format stipulated above.** Your marks may be affected if your tutor cannot open or properly view your submission.
- **Do not leave submission to the very last minute**. Always allow time in case of technical issues. Also, save your work frequently to avoid losing your work at the last minute.

- The date and time of your submission is taken from the Blackboard server and is recorded when your submission is complete, not when you click Submit.
- **It is essential that you check that you have submitted the correct file(s),** and that each completed file was received. Submission receipts are accessed from the Coursework tab.
- UWE academic regulations for late submissions will be applied. Please check module handbook.

***Note: All submitted work will be checked for Plagiarism using SafeAssign or other suitable software/approaches. Every student undertaking an assignment or other piece of assessed work is required to take, and will be deemed to have taken, full responsibility for all the work submitted by the student. In particular, this includes responsibility for any assessment offence (e.g., Plagiarism, collusion) committed. Assessment offence such as plagiarism or collusion will result in 0 marks.***

**Where should I start?**

Read the assessment task carefully, discuss with your group members and lab tutors. If there is any ambiguity, then clarify by discussing to your lab tutors. Your lab tutors will play role of representative of the case study. If you have questions about this assignment or if you would like to clarify requirements, then please discuss with your lab tutors during your practical lab sessions or post them on Discussion Board in Blackboard. Apply weekly learning on your project i.e., start with a design, improve design as understanding grows, decide on software methodology, start development process by taking design into consideration. Here is an indicative list of weekly milestones that you may adapt:

Weeks 1-3: Coursework is released, and you have read and understood the project brief. You have clarified any questions with your tutors and have started planning your coursework.

Weeks 4-6: You have begun software design and selected a software methodology for the project.

Weeks 7–8: You have begun software development by focusing on the design. You may need to revisit your design once development starts.

Weeks 9-10: You have completed the design, development & testing.

Week 11: Your project is completed, and you have started to review and refine the design/code/testing. You are ready to submit the coursework.

Week 12: Demo Week.

**What do I need to do to pass?**

To pass this assessment, you must achieve at least 40% of the marks. Refer to the marking criteria for detailed information on the assessment's grading**. You must pass both the portfolio and the viva/demo to pass the module**. See Task 1 and Task 2 for further details. Do not leave project work until the last few weeks. Work on a weekly basis and submit your work before the submission deadline.

**How do I achieve high marks in this assessment?**

To achieve higher marks, ensure you address all aspects of the assessment. The quality and appropriate level of detail in your completed work will lead to better scores. This can be accomplished by actively engaging with the module activities. Additionally, have your work reviewed by your lab tutors to receive formative feedback and improve your submission.

**How does the learning and teaching relate to the assessment?**
Working on the portfolio will help you to develop effective skills required for software design and development. Teaching block 1 focuses on software design (which also include choosing a software methodology for your project) whilst Teaching block 2 focuses on development and testing. Your lab tutor will play role of representative of the given case study. Please do not send emails to your tutors about the assignment. If you have questions about this assignment or if you would like to clarify requirements, then please discuss with your lab tutors during your practical lab sessions or post it on the Discussion Board in the Blackboard and use the forum under the title "Portfolio Queries".

**What additional resources may help me complete this assessment?**

- Your main source will be learning material on the Blackboard.
- In addition, UWE library study skills pages can be useful i.e.,
    - https://www.uwe.ac.uk/study/study-support/study-skills
    - How to give a presentation
- Your practical sessions to work on practical exercises and gain formative feedback. Practical sessions will encourage you to work in groups which will also help you in this coursework. You will also get opportunity to learn from other groups during practical sessions.
- You can use Blackboard discussion board to post queries for your lab tutors or discuss specific topics with other students

**What do I do if I am concerned about completing this assessment?**
It is recommended that you review all the relevant materials on Blackboard. You can also speak to your module leader for advice and guidance.
UWE Bristol offer a range of Assessment Support Options that you can explore through this link, and both Student Support Advisers and Wellbeing Support are available.
For further information, please see the Student study essentials.

**How do I avoid an Assessment Offence on this module?**
Use the support above if you feel unable to submit your own work for this module.

UWE Bristol's UWE's Assessment Offences Policy (Academic integrity - Assessments | UWE Bristol) requires that you submit work that is entirely your own and reflects your own learning. It is important to:

Ensure that you reference all sources used, using the UWE Bristol Harvard - Referencing | UWE Bristol. Use the guidance available on UWE's Study Skills referencing pages( Referencing - Study skills | UWE Bristol ).

Avoid copying and pasting any work into this assessment, including your own previous assessments, work from other students or internet sources.

## Marks and Feedback

**Portfolio carries 75% weight, and Demo/Viva carries 25% weight.**
It is common practice with all UWE standard undergraduate assessments, the pass mark to complete the module is 40%. Similarly, for this assessment (both Task 1 and Task 2), the pass mark is 40% for each task. Your assessment will be evaluated based on the marking criteria outlined in the rubrics (see marking criteria).

**You can use these to evaluate your own work before you submit.**

1. UWE Bristol's Academic Conduct Policy and Academic Misconduct Procedures (Assessment Offence Policy) is available from the Academic Integrity webpages and requires that you submit work that is entirely your own and reflects your own learning, so it is important to:
    - Ensure you reference all sources used, using the UWE Harvard/OSCOLA system (amend as appropriate) and the guidance available on UWE's Study Skills referencing pages.
    - Refer to peer reviewed primary sources, rather than using AI or secondary sources
    - Avoid copying and pasting any work into this assessment, including your own previous assessments, work from other students or internet sources
    - Develop your own style, arguments and wording, so avoid copying sources and changing individual words but keeping, essentially, the same sentences and/or structures from other sources
    - Never give your work to others who may copy it
    - If an individual assessment, develop your own work and preparation, and do not allow anyone to make amends on your work (including proof-readers, who may highlight issues but not edit the work).

    **When submitting your work, you will be required to confirm that the work is your own,** and text-matching software and other methods are routinely used to check submissions against other submissions to the university and internet sources. Details of what constitutes plagiarism and how to avoid it can be found on UWE's Study Skills pages about avoiding plagiarism.

## Marking Criteria

| Portfolio (75 marks) | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | **0.0-3.0** | **3.1-4.0** | **4.1-5.0** | **5.1-6.0** | **6.1-7.0** | **7.1-8.5** | **8.6-10.0** | **Mark & Feedback** |
| **Use case diagram (10)** | No or mostly incorrect diagram (less than 30%). | Partial diagram, some actors / use-cases correct (less than 40%). | Most elements correct but partial diagram, (less than 50%). | Some incorrect elements and almost complete diagram (less than 60%). | Mostly correct elements and nearly complete diagram (less than 70%). | Some minor errors, but fully complete diagram (less than 85%). | Fully complete and correct, actors and/or use-cases / relationships are correct (100% complete). | |
| **Class diagram (10)** | No or mostly incorrect Class diagram (less than 30%). | Partial diagram with some Class naming convention, relationships, attributes / methods of the classes and their visibility and types are only partially correct (less than 40%). | Partial diagram and Class naming, relationships, attributes / methods of the classes and their visibility and types are mostly correct (less than 50%). | Some elements still incorrect but almost complete, Class naming convention, relationships, attributes / methods of the classes and their visibility and types are partially correct (less than 60%). | Mostly correct but nearly complete diagram, Class naming convention, relationships, attributes / methods of the classes and their visibility and types are correct (less than 70%). | Complete diagram but few minor issues, Class naming convention, relationships, attributes / methods of the classes and their visibility and types are partially correct (less than 85%). | Complete and accurate, Class naming convention, relationships, attributes / methods of the classes and their visibility and types are correct (100% complete) | |
| | **0.0-4.5** | **4.6-6.0** | **6.1-7.5** | **7.6-9.0** | **9.1-10.5** | **10.6-12.75** | **12.76-15.0** | |
| **Sequence diagram (at least three diagrams) (15)** | Missing or mostly incorrect (less than 30%). | Partial diagrams, with major errors in lifelines, messages, or activation (less than 40%). | Partial diagrams, mostly correct, lifelines, activation bars, and messages are | Nearly complete, but with noticeable errors, lifelines, activation bars, and messages | Nearly complete and mostly correct, lifelines, activation bars, and messages are correct (less than 70%). | Complete diagrams but some minor issues, lifelines, activation bars, and messages are | Fully complete and accurate, lifelines, activation bars, and messages are | |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | correct (less than 50%). | are only partially correct (less than 60%). | | only partially correct (less than 85%). | correct (100% complete). | |
| | **0.0-9.0** | **9.1-12.0** | **12.1-15.0** | **15.1-18.0** | **18.1-21.0** | **21.1-25.5** | **25.6-30.0** | |
| **Implementation (30)** | Little or no development. design not followed (less than 30%). | Basic development with minimal mapping to design. App runs but GUI and messages not accurate (less than 40%). | Partially developed. App runs with GUI showing input/output with some errors (less than 50%). | Some business logic developed, runs but GUI still partly correct, input and output messages only partially correct (less than 60%). | Mostly complete business logic developed, but some minor GUI or feature issues remains. Input and output messages only partially correct (less than 70%). | Fully developed with security. Some minor issues in GUI or features. Input and output messages are almost correct. (less than 85%). | Complete, accurate implementation with correct GUI, security, scalability and database integration (100% complete). | |
| | **0.0-3.0** | **3.1-4.0** | **4.1-5.0** | **5.1-6.0** | **6.1-7.0** | **7.1-8.5** | **8.6-10.0** | |
| **Testing (10)** | Few or no tests done (less than 30% tested). | Some testing done, (less than 40% tested). | Moderate testing, (less than 50% tested). | Fair level of testing, (less than 60% tested). | Almost complete testing, (less than 70% tested). | Well-tested, minor gaps, (less than 85% tested). | Fully tested with all scenarios covered, (100% tested). | |

| **Demo/Viva (25 marks)** | | | |
|---|---|---|---|
| **Demo (15)** | Absent<br>Inadequate demo preparation (barely able to explain the methodology, design, coding/testing and/or work done)<br>Good demonstration (good explanation of methodology, design, coding/testing and/or work done)<br>Very Good demonstration (very good explanation of methodology, design, coding/testing and/or work done)<br>Excellent demonstration (excellent explanation of methodology, design, coding/testing and/or work done) | 0.0<br>1.0- 4.0<br>4.1- 8.0<br>8.1- 12.0<br>12.1- 15.0 | |
| **Individual Q&A (10)** | Absent<br>Inadequate (barely able to explain the methodology, analysis/design, coding and/or work done)<br>Good (good explanation of methodology, analysis/design, coding and/or work done)<br>Very Good (very good explanation of methodology, analysis/design, coding and/or work done)<br>Excellent (excellent explanation of methodology, analysis/design, coding and/or work done) | 0.0<br>0.0- 3.0<br>3.1- 6.0<br>6.1- 8.0<br>8.1- 10.0 | |